GR02 - Modelo Relacional

Tema 2 · El Modelo Relacional — Guía de repaso

> **Objetivo de esta guía**: repasar en poco tiempo los conceptos esenciales del modelo relacional. Cada sección va de la idea intuitiva al detalle formal, con ejemplos mínimos. Al final tienes un glosario-chuleta y preguntas de autoevaluación.

---

0. ¿De dónde viene todo esto?

El modelo relacional lo propuso **E.F. Codd** (IBM Research) en 1970 en el artículo *A Relational Model for Large Shared Data Banks*. Revolucionó la gestión de bases de datos y le valió el **Premio Turing**. Su fuerza está en que se apoya en una base matemática sólida: la **teoría de conjuntos**.

Ten en cuenta desde el principio que existen **dos modelos**:

- El **modelo formal** (matemático), que estudiamos en teoría.
- El **modelo práctico**, basado en **SQL**, que implementan los SGBDR reales. Hay diferencias importantes entre ambos (p. ej., SQL permite filas duplicadas; el modelo formal no).

---

1. Las estructuras: relación, tupla, atributo, dominio

1.1. La idea intuitiva: una tabla

Una **relación** se parece a una **tabla**: filas con datos y columnas con encabezados. Pero cada concepto informal tiene su nombre formal:

Informal (tabla)	Formal (modelo relacional)
Tabla	**Relación**
Nombre de columna	**Atributo**
Tipo de columna	**Dominio**
Fila	**Tupla**
Definición de tabla	**Esquema de relación**
Datos de la tabla	**Estado de la relación**

> ⚠️ Que una relación *parezca* una tabla no significa que se almacene físicamente así. La implementación física es asunto del SGBD.

1.2. La definición formal: subconjunto de un producto cartesiano

Una **relación** es un **subconjunto del producto cartesiano** de uno o más conjuntos (los dominios):

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

Cada elemento de R es una **tupla** $(d_1, d_2, \ldots, d_n)$ con $d_i \in D_i$.

Ejemplo mínimo. Con dos dominios pequeños:

- $dom(A_1) = \{0, 1\}$
- $dom(A_2) = \{a, b, c\}$

El producto cartesiano contiene las 6 combinaciones posibles: {(0,a), (0,b), (0,c), (1,a), (1,b), (1,c)}. Un posible **estado** de la relación es el subconjunto {(0,a), (0,b), (1,c)}: tres tuplas de grado 2.

Ejemplo con significado. La relación ASIGNATURA(nombre, curso, créditos) es un subconjunto de todas las combinaciones posibles de nombres × cursos × créditos, pero solo contiene las tuplas que representan asignaturas reales:

```
ASIGNATURA = {
  ("Bases de Datos", 2, 6),
  ("Programación", 1, 6),
  ("Redes", 3, 6)
}
```

1.3. Vocabulario esencial

- **Esquema de relación**: $R(A_1, A_2, \ldots, A_n)$. Es la *estructura*: nombre de la relación + sus atributos. Ej.: $CIENE(id, nombre, dirección, teléfono)$.
- **Grado**: número de atributos. CLIENTE tiene grado 4.
- **Estado (o instancia)**: el conjunto *actual* de tuplas, $r(R)$. Cambia cada vez que se modifican los datos; el esquema, no.
- **Cardinalidad**: número de tuplas del estado actual.
- **Dominio**: conjunto de valores válidos de un atributo. Tiene una definición lógica ("teléfonos válidos de 9 dígitos") y un tipo de datos/formato asociado. Un mismo dominio puede usarse en varios atributos con papeles distintos: el dominio *Fecha* puede dar lugar a los atributos *fechaFactura* y *fechaPago*.

- **Tupla**: conjunto **ordenado** de valores, uno por atributo. Ej.: (632895, 'John Smith', '101 Main St.', '600123456').

1.4. Características de las relaciones

1. **Las tuplas NO están ordenadas.** Una relación es un conjunto: no hay "primera fila" ni "última fila", aunque las dibujemos en forma de tabla.
2. **No hay tuplas duplicadas.** Dado que una relación es un conjunto, todos sus elementos son distintos: dos tuplas no pueden coincidir en *todos* sus valores.
3. **Los atributos (y los valores dentro de la tupla) SÍ están ordenados** en la definición formal $R(A_1, \ldots, A_n)$.
4. Los valores de una tupla deben ser **atómicos** (indivisibles): nada de listas o conjuntos como valor.
5. Se admite el valor **NULL** para representar información desconocida, no disponible o inaplicable... pero conviene **evitarlo** en la medida de lo posible al diseñar.

1.5. Notación que debes reconocer

- $R, S,$ (mayúsculas): nombres de relaciones. $r, s,$ (minúsculas): estados.
- $t, u,$: tuplas.
- $R.A$ (notación de punto): atributo A de la relación R. Necesaria porque dos relaciones distintas pueden repetir nombre de atributo (dentro de una misma relación, no).
- $t[A]$ o $t.A$: valor del atributo A en la tupla t.
- $t[A_u, A, \ldots]$: subtupla con los valores de esos atributos.

Ejemplo. Dada t = ('Bárbara Benson', '123456789', '(817)839-8461', '7384 Fontana Lane', NULL, 19, 3.25) de ESTUDIANTE:

- $t[nombre]$ = ('Bárbara Benson')
- $t[dni, notaMedia, edad]$ = ('123456789', 3.25, 19)

Convención tipográfica: nombres de relación en Mayúscula, atributos en minúscula. En el esquema, la clave primaria se subraya y las claves alternativas se resaltan.

1.6. Base de datos relacional

- **Esquema de BD relacional** S: un conjunto de esquemas de relación $\{R_1, \ldots, R_n\}$ **más** un conjunto RI de restricciones de integridad.
- **Estado de BD**: un estado por cada relación, $DB = \{r_1, \ldots, r_m\}$, que **satisface todas las restricciones** de RI. Si no las satisface, el estado es **no válido o inconsistente**.
- Solo hay 3 operaciones básicas para cambiar el estado: **INSERTAR**, **BORRAR** y **MODIFICAR** tuplas. Esta limitación es una *fortaleza*: permite describir operaciones a muy

alto nivel y que el SGBD las implemente eficientemente.

Nuestra BD de referencia (EMPRESA):

- EMPLEADO(nombre, apellido1, apellido2, dni, fechaNac, dirección, sexo, sueldo, supervisor, dpto)
- DEPARTAMENTO(nombre, numero, director, fechaIngresoDirector)
- LOCALIZACIONES_DPTO(dpto, ubicacion)
- PROYECTO(nombre, numero, ubicacion, dpto)
- TRABAJA_EN(empleado, proyecto, horas) ← representa una *asociación* entre entidades
- FAMILIAR(empleado, nombre, sexo, fechaNac, relación)

---

2. Las restricciones: qué estados son válidos

Las restricciones son **condiciones que deben cumplirse en todos los estados válidos** de la BD. Vienen dadas por las reglas del minimundo que representamos.

2.1. Los tres grandes tipos

Tipo	¿Dónde se define?	Ejemplo
Implícitas (inherentes)	En el propio modelo	No se permiten tuplas duplicadas ni valores no atómicos
Explícitas (basadas en esquema)	En el esquema, con el DDL	Dominio, clave, integridad de entidad, integridad referencial
Semánticas (de aplicación)	En los programas (el modelo no llega)	"Un empleado no puede superar 56 h/semana entre todos sus proyectos"

Nos centramos en las **explícitas**, que son cuatro. 👇

2.2. Restricciones de dominio

Cada valor de una tupla debe ser **atómico** y **pertenecer al dominio de su atributo** (o ser NULL si el atributo lo permite).

En SQL, el dominio se define con el **tipo de datos** de la columna, y se puede afinar con:

- **Subrangos**: `CHECK (sueldo > 0)`
- **Enumerados**: `CHECK (sexo IN ('M', 'F', 'O'))`
- **NOT NULL** para prohibir nulos.

```
sexo   CHAR(1)       NOT NULL CHECK (sexo IN ('M','F','O')),  -- enumerado
sueldo NUMERIC(12,2) NOT NULL CHECK (sueldo > 0)              -- subrango
```

2.3. Restricciones de clave

Esta jerarquía de conceptos es fundamental. De más general a más específico:

1. **Superclave (SK)**: conjunto de atributos que **nunca se repite** entre tuplas (propiedad de unicidad): para tuplas distintas t_1, t_2: $t_1[S] \neq t_2[S]$.
2. **Clave (candidata)**: superclave **mínima**: si le quitas cualquier atributo, deja de ser superclave.
3. **Clave primaria (PK)**: la clave candidata que *elegimos* para identificar las tuplas. Se subraya.
4. **Claves alternativas**: las candidatas no elegidas. Se resaltan.
5. **Atributo primo**: atributo que pertenece a *alguna* clave candidata.

Ejemplo clásico. $COCE$ (estado, matricula, numSerie, marca, modelo, año)

- Claves candidatas: {estado, matricula} y {numSerie}
- PK elegida: {numSerie}. Alternativa: {estado, matricula}
- Atributos primos: estado, matricula, numSerie
- {numSerie, marca} es superclave pero **no** clave (no es mínima)

Recuerda:

- Toda clave es superclave; no toda superclave es clave.
- Cualquier conjunto que *contenga* una clave es superclave.

> 💡 **Regla práctica**: elige como PK la clave candidata más pequeña. En entornos profesionales se suelen usar **claves sustitutas** (*surrogate keys*): enteros autoincrementales sin significado de negocio, que nunca necesitan cambiar.

En SQL: `PRIMARY KEY` para la primaria, `UNIQUE` para las alternativas.

```
CREATE TABLE DEPARTAMENTO (
    nombre  VARCHAR(40) UNIQUE NOT NULL,   -- clave alternativa
    numero  INTEGER PRIMARY KEY,           -- PK (implica NOT NULL)
    director VARCHAR(10),
    fechaIngresoDirector DATE NOT NULL
);

-- PK compuesta (varias columnas):
CREATE TABLE LOCALIZACIONES_DPTO (
```

```
    dpto      INTEGER NOT NULL,
    ubicacion VARCHAR(40) NOT NULL,
    PRIMARY KEY (dpto, ubicacion)
);
```

2.4. Integridad de entidad

Ningún atributo de la PK puede ser NULL en ninguna tupla. Motivo: la PK identifica cada tupla; con NULL no podría hacerlo. Si la PK es compuesta, la prohibición afecta a *todos* sus atributos.

2.5. Integridad referencial

Es la única restricción **entre dos relaciones**: la **referenciante** R_1 (que contiene la **clave externa**, FK) y la **referenciada** R_2 (cuya PK es apuntada).

La regla: el valor de la FK en R_1 debe ser

1. un valor de PK **que exista** en R_2, o
2. **NULL** (y en ese caso la FK no puede formar parte de la PK de R_1).

Ejemplo en EMPRESA:

- EMPLEADO.dpto → DEPARTAMENTO.numero (todo empleado pertenece a un departamento existente; relación N:1)
- EMPLEADO.supervisor → EMPLEADO.dni (¡una FK puede apuntar a la *misma* relación!). Admite NULL: hay empleados sin supervisor.
- DEPARTAMENTO.director → EMPLEADO.dni

```
CREATE TABLE EMPLEADO (
    ...
    PRIMARY KEY (dni),
    FOREIGN KEY (supervisor) REFERENCES EMPLEADO (dni),
    FOREIGN KEY (dpto)       REFERENCES DEPARTAMENTO (numero)
);
```

2.6. Restricciones semánticas

Reglas del negocio que el modelo relacional **no puede expresar** con las estructuras anteriores. SQL fue ganando expresividad: SQL-99 introdujo `CREATE TRIGGER` y `CREATE ASSERTION` para cubrir parte de ellas.

---

3. Operaciones de actualización y sus peligros

Tres operaciones: **INSERT**, **DELETE**, **UPDATE**. Ninguna debe dejar la BD en un estado que viole las restricciones. La pregunta clave de esta sección: *¿qué restricciones puede violar cada operación y qué hace el SGBD cuando ocurre?*

3.1. ¿Qué puede violar cada operación?

Operación	Puede violar...
INSERT	**Todas**: dominio, clave, integridad de entidad e integridad referencial
DELETE	**Solo integridad referencial** (si la tupla borrada estaba referenciada por otras)
UPDATE	Depende del atributo: si tocas la **PK** → como borrar+insertar; si tocas una **FK** → integridad referencial; si tocas un atributo ordinario → dominio / NOT NULL / UNIQUE

Ejemplo rápido (INSERT en EMPLEADO):

```
-- Viola integridad de entidad: dni (PK) es NULL
INSERT INTO EMPLEADO VALUES ('Cecilia','Santos','García', NULL, ...);

-- Viola clave: ya existe un empleado con dni '999887777'
INSERT INTO EMPLEADO VALUES ('Alicia','Jiménez','Celaya','999887777', ...);

-- Viola integridad referencial: no existe el departamento 7
INSERT INTO EMPLEADO VALUES ('Cecilia','Santos','García','677678989', ..., 7);
```

3.2. Acciones ante una violación de integridad referencial

Al definir cada FK, el diseñador **debe decidir** qué hacer cuando se borra o modifica la PK referenciada:

Opción	Qué hace
RESTRICT	Rechaza la operación (verifica *antes* de ejecutarla)
NO ACTION	Permite la operación y verifica al final; si hay violación, deshace. Opción por defecto en muchos SGBD
CASCADE	Propaga el cambio a las tuplas referenciantes (borrado o actualización en cadena)
SET NULL	Pone a NULL la FK de las tuplas referenciantes (la FK debe admitir NULL)

Opción	Qué hace
SET DEFAULT	Pone la FK a su valor por defecto (debe estar definido y ser válido)

```
FOREIGN KEY (supervisor) REFERENCES EMPLEADO (dni)
    ON DELETE SET NULL     ON UPDATE CASCADE,
FOREIGN KEY (dpto) REFERENCES DEPARTAMENTO (numero)
    ON DELETE SET DEFAULT  ON UPDATE CASCADE
```

Estas decisiones son **decisiones de diseño con consecuencias sobre los datos**: p. ej., si usas `SET DEFAULT` con `dpto DEFAULT 1`, el departamento 1 no debe poder borrarse nunca.

> ⚠️ **Sobre modificar claves primarias**: aunque `ON UPDATE CASCADE` existe, la práctica profesional es **evitar modificar las PK** (usar RESTRICT/NO ACTION). Una PK debe ser un identificador único e *inmutable*; las actualizaciones en cascada son costosas y arriesgadas. La verdadera solución: usar claves sustitutas que nunca necesiten cambiar.

---

4. Las 12 Reglas de Codd (1985): en versión mínima

Criterios formulados por Codd para evaluar si un SGBD es *verdaderamente* relacional. No hace falta memorizarlas palabra a palabra, pero sí entender su espíritu:

- **Regla 0 (fundamental)**: todo debe gestionarse mediante capacidades relacionales.
- **Reglas 1-2 (información y acceso)**: toda la información se representa como valores en tablas, y todo dato es accesible mediante tabla + PK + columna.
- **Regla 3 (nulos)**: tratamiento sistemático del NULL (distinto de 0, cadena vacía o blancos).
- **Regla 4 (catálogo)**: los metadatos (descripción de la BD) se consultan como datos normales, con el mismo lenguaje.
- **Regla 5 (sublenguaje completo)**: debe existir un lenguaje que cubra definición, manipulación, seguridad, integridad y transacciones (→ SQL).
- **Reglas 6-7 (vistas y alto nivel)**: las vistas teóricamente actualizables deben serlo; insertar/actualizar/borrar operan sobre relaciones completas, no fila a fila.
- **Reglas 8-9 (independencia física y lógica)**: los cambios de almacenamiento o de estructura lógica que preservan la información no rompen las aplicaciones.
- **Regla 10 (independencia de integridad)**: las restricciones se definen en el lenguaje relacional y se guardan en el catálogo, no en los programas.
- **Regla 11 (independencia de distribución)**: al usuario le da igual si la BD está en un servidor o distribuida por el mundo.

- **Regla 12 (no subversión)**: ningún lenguaje de bajo nivel puede saltarse las restricciones definidas a alto nivel.

En la práctica, **ningún SGBDR comercial las cumple al 100%** (Oracle, MySQL, SQL Server, PostgreSQL, SQLite las cumplen en distinto grado), pero siguen siendo la referencia conceptual del modelo.

📌 Glosario-chuleta

Concepto	En una frase
Relación	Subconjunto del producto cartesiano de dominios; intuitivamente, una tabla
Esquema	Estructura: $R(A_1, \ldots, A_n)$; no cambia con los datos
Estado / instancia	Conjunto actual de tuplas $r(R)$; cambia con cada actualización
Grado / cardinalidad	Nº de atributos / nº de tuplas
Dominio	Valores válidos de un atributo (definición lógica + tipo de datos)
Tupla	Fila: valores ordenados y atómicos, uno por atributo
Superclave	Conjunto de atributos con unicidad garantizada
Clave candidata	Superclave mínima
PK / alternativa	Candidata elegida / candidatas restantes
Atributo primo	Pertenece a alguna clave candidata
Integridad de entidad	La PK no admite NULL
Integridad referencial	Toda FK apunta a una PK existente o es NULL
FK (clave externa)	Atributo(s) de una relación que referencian la PK de otra (o de sí misma)
RESTRICT / CASCADE / SET NULL / SET DEFAULT	Acciones posibles al violar la integridad referencial

---

✅ Autoevaluación rápida

1. ¿Cuál es la diferencia entre *esquema* y *estado* de una relación?

2. ¿Por qué una relación no puede tener tuplas duplicadas?
3. En $COCE$(estado, matricula, numSerie, marca, modelo, año), ¿por qué {numSerie, marca} es superclave pero no clave?
4. ¿Puede un atributo de la clave primaria valer NULL? ¿Y una clave externa?
5. ¿Qué restricciones puede violar un INSERT? ¿Y un DELETE?
6. Si borramos un departamento referenciado por empleados, ¿qué ocurre con `ON DELETE SET NULL`? ¿Qué requisito debe cumplir la FK?
7. ¿Por qué se desaconseja `ON UPDATE CASCADE` sobre claves primarias y cuál es la alternativa profesional?
8. ¿Qué es una clave sustituta (*surrogate key*) y qué ventaja tiene?
9. Cita dos restricciones *implícitas* del modelo relacional.
10. ¿Qué dice la regla de la no subversión (regla 12 de Codd)?

(Pista: todas las respuestas están en esta guía. Si dudas en alguna, vuelve a la sección correspondiente de los apuntes completos.)